Succinct Non-Interactive Zero Knowledge for a von Neumann Architecture

Scenario

Setup

- client owns public input x (query, for example)
- server owns private input w (Database, for example)
- program F known to client and server
- client wishes to learn $z := F(x, w)$

Challenges

- client wants to be certain that the server reports the correct output z
- server does not want to reveal anything about w

Solution

- server acting as prover and the client as verfier
- server proves that "there exists such a w such that $z := F(x, w)$
- soundness guarantees that if no such w exists, prover caanot convince verifier
- zero-knowledge guarantees that verifier cannot learn anything about w

Efficiency

- proof non interactive, so client only sends input x and recieves $\tilde{z}$ together with proof string π

Goal

- construct a zk-SNARK implementation supporting executions on a universal von Neumann RISC machine

Circuit Generator

- input bounds: l,n,T
- l := instructions
- n := input size
- T := calculation steps

improvements

- circuit generator is universal:
  - produces circuit that only needs at max l,n,T steps to verify correctness

  - only needs to expensively generate one pair of keys to verify all programs to a given size (proving key, verfication key)
  - Supports von Neumann RISC architecture (called vnTinyRam)
  - size of generated circuit $C_{l,n,T}$ grows additive on program size

construction

- Ensure via dedicated circuit that Program P is loaded correctly into initial state of the memory
- validate that the instruction fetches
- validate instruction execution
- vaidate the data loads/stores
- execution trace $tr = (S_1, I_1, \dots, S_T, I_T)$
  - S_i := i-th CPU-State (contains pc, registers, flags at given time Stamp i)
  - I_i := i-th executed instruction

1. verify inital load of Program P into memory

- extend execution trace tr wit initial boot section where P is loaded into memory for instruction per instruction

2. verify validity of instruction fetches and memory accesses

- $tr = (A_1, \dots, A_{2T})$
- sorted list of all memory accesses (since instruction fetches is a special form of memory access)
- design C to verify $tr* = MemSort(tr)$ is a correct permutation of tr
  - $Memsort(tr)$ is a sorting of tr by accessed memory addresses
- create circuit C'_{mem} to ensure the load-after-load and load-after-store consistency

3. verify validity of instruction execution

- Construct cirtcuit C_{exe} such
- given two CPU States S, S' and instruction I
- $C_{exe}(S, S', I, g)$ is only satisfied for a guess g iff S' can be reached from S by executing I for some memory state
- g is a guess for a memory state

Conclusion

- key Generator G gets the circuit C and security parameter λ as input and outputs proving key (pk) and verification key (vk)
- prover uses pk, public input x and secret witness w to generate proof π
- verifier uses vk, public input x and proof π to verify correctness of π

- zk-SNARK verifies if there exists a such a witness w for public input x that satisfies the generated circuit C